EN3160 Assignment 3 on Neural Networks

A.A.D.K. Amarasinghe

230033J

Q1

In [1]:
```
# Q1
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
```

In [2]:
```
# 1. Dataloading
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

batch_size = 50
```

In [3]:
```
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=False, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                          shuffle=True, num_workers=2)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=False, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                         shuffle=False, num_workers=2)
```

In [ ]:
```
classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')

# 2. Define Network Parameters
Din = 3 * 32 * 32
H = 100            # Hidden Dense Layers
K = 10
std = 1e-2

# Initialize weights and biases
w1 = torch.randn(Din, H) * std
b1 = torch.zeros(H)
w2 = torch.randn(H, K) * std
b2 = torch.zeros(K)
```

In [ ]:
```
# Hyperparameters
epochs = 10
lr = 1e-2
lr_decay = 0.9
```

```
reg = 0
loss_history = []
```

In [ ]:
```
# 3. Training Loop
for t in range(epochs):
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # Get inputs and labels
        inputs, labels = data
        Ntr = inputs.shape[0]
        x_train = inputs.view(Ntr, -1)
        y_train_onehot = nn.functional.one_hot(labels, K).float()

        # Forward pass
        h = torch.sigmoid(x_train.mm(w1) + b1)
        y_pred = h.mm(w2) + b2
        log_probs = torch.log_softmax(y_pred, dim=1)

        # Loss calculation (Cross-Entropy with regularization)
        loss = -(1 / Ntr) * torch.sum(y_train_onehot * log_probs) \
               + reg * (torch.sum(w1 ** 2) + torch.sum(w2 ** 2))
        loss_history.append(loss.item())
        running_loss += loss.item()

        # Backpropagation
        dy_pred = (torch.exp(log_probs) - y_train_onehot) / Ntr  # softmax + CE
        dw2 = h.t().mm(dy_pred) + reg * w2
        db2 = dy_pred.sum(dim=0)
        dh = dy_pred.mm(w2.t())
        dz1 = dh * h * (1 - h)
        dw1 = x_train.t().mm(dz1) + reg * w1
        db1 = dz1.sum(dim=0)

        # Parameter update
        w1 -= lr * dw1
        b1 -= lr * db1
        w2 -= lr * dw2
        b2 -= lr * db2

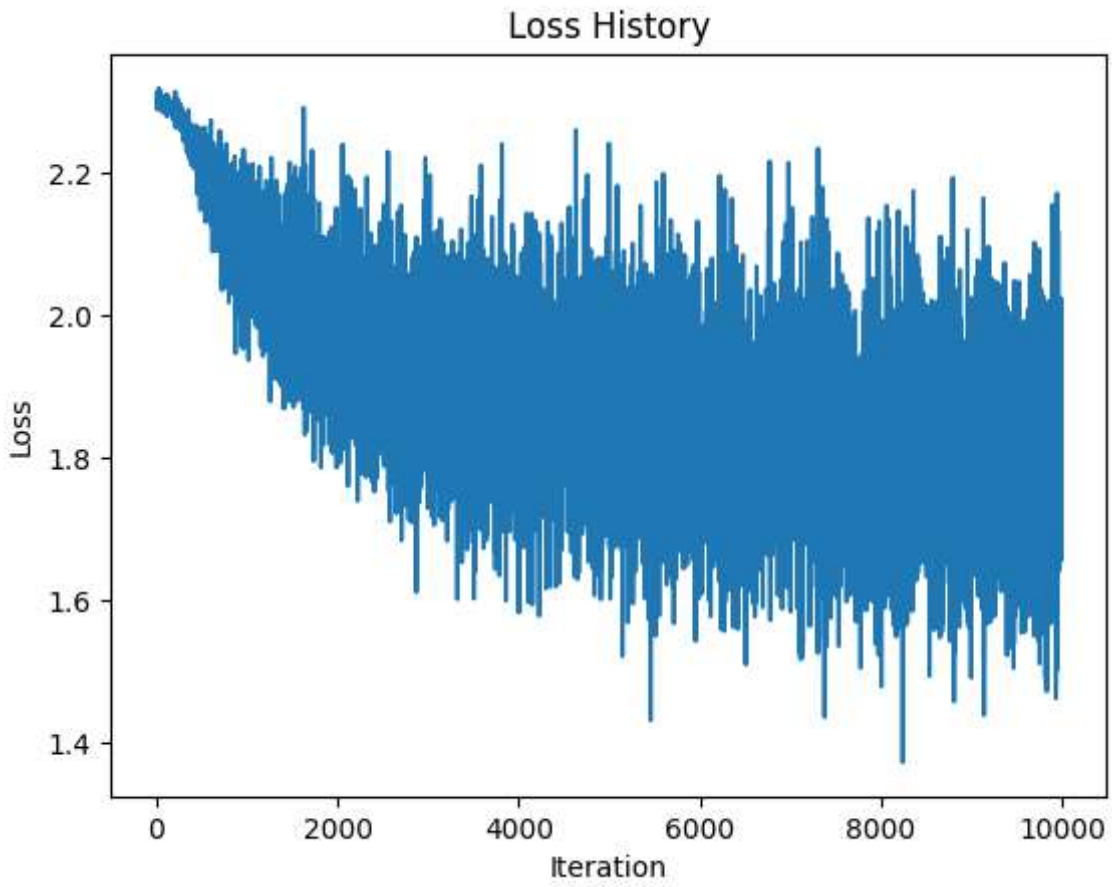
    # Print loss for every epoch
    if t % 1 == 0:
        print(f"Epoch {t + 1}/{epochs}, Loss: {running_loss / len(trainloader)}")

    # Learning rate decay
    lr *= lr_decay
```

```
Epoch 1/10, Loss: 2.2062191746234894
Epoch 2/10, Loss: 2.019605132818222
Epoch 3/10, Loss: 1.9426423196792602
Epoch 4/10, Loss: 1.902088950276375
Epoch 5/10, Loss: 1.8736447871923447
Epoch 6/10, Loss: 1.851162710905075
Epoch 7/10, Loss: 1.8329045095443726
Epoch 8/10, Loss: 1.818039428949356
Epoch 9/10, Loss: 1.805706608057022
Epoch 10/10, Loss: 1.7955967390537262
```

In [7]:
```
# 4. Plotting the Loss History
plt.plot(loss_history)
plt.title("Loss History")
```

```
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.show()
```



In [8]:
```
# 5. Calculate Accuracy on Training Set
correct_train = 0
total_train = 0
with torch.no_grad():
    for data in trainloader:
        inputs, labels = data
        Ntr = inputs.shape[0]
        x_train = inputs.view(Ntr, -1)

        # Forward pass
        h = torch.sigmoid(x_train.mm(w1) + b1)
        y_train_pred = h.mm(w2) + b2
        predicted_train = torch.argmax(y_train_pred, dim=1)
        total_train += labels.size(0)
        correct_train += (predicted_train == labels).sum().item()
train_acc = 100 * correct_train / total_train
print(f"Training accuracy: {train_acc:.2f}%")
```

```
Training accuracy: 36.96%
```

In [9]:
```
# 6. Calculate Accuracy on Test Set
correct_test = 0
total_test = 0
with torch.no_grad():
    for data in testloader:
        inputs, labels = data
        Nte = inputs.shape[0]
        x_test = inputs.view(Nte, -1)
```

```
        # Forward pass
        h = torch.sigmoid(x_test.mm(w1) + b1)
        y_test_pred = h.mm(w2) + b2
        predicted_test = torch.argmax(y_test_pred, dim=1)
        total_test += labels.size(0)
        correct_test += (predicted_test == labels).sum().item()
test_acc = 100 * correct_test / total_test
print(f"Test accuracy: {test_acc:.2f}%")
```

```
Test accuracy: 37.09%
```

This is a fully connected neural network with one hidden layer trained on CIFAR-10 with manually coded forward and backward passes.

Input (3072) -> Hidden (100, sigmoid) -> Output (10) -> Softmax

Changes from the single dense layer network

1. Added a hidden layer with 100 nodes and sigmoid activation so the model can learn non-linear patterns instead of only a linear mapping. Backpropagation now passes the error through the hidden layer using the chain rule.
2. Replaced Mean Squared Error with Cross-Entropy loss (with softmax) which suits classification better. The output gradient simplifies to (softmax - onehot) / N.
3. Trained for 10 epochs with the initialization std raised to 1e-2 and the learning rate raised to 1e-2 to suit the new model. Training accuracy 36.96%, test accuracy 37.09%.

The close train and test accuracies show no overfitting. The accuracy is low because the model ignores spatial image structure and the training is short so its better to use model like CNN than deep learning model.

Q2

In [1]:
```
# Q2
import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

In [2]:
```
SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

norm = transforms.Normalize((0.1307,), (0.3081,))

train_tf = transforms.Compose([
    transforms.RandomAffine(degrees=10, translate=(0.1, 0.1), scale=(0.9, 1.1)),
```

```
    transforms.ToTensor(), norm])
eval_tf = transforms.Compose([transforms.ToTensor(), norm])
```

```
Using device: cpu
```

In [3]:
```
train_set = torchvision.datasets.MNIST('./data1', train=True, download=False, tr
train_eval_set = torchvision.datasets.MNIST('./data1', train=True, download=Fals
test_set = torchvision.datasets.MNIST('./data1', train=False, download=True, tra

trainloader = torch.utils.data.DataLoader(train_set, batch_size=64, shuffle=True
train_eval_loader = torch.utils.data.DataLoader(train_eval_set, batch_size=1000,
testloader = torch.utils.data.DataLoader(test_set, batch_size=1000, shuffle=Fals
```

In [4]:
```
class LeNet5(nn.Module):  # architecture unchanged (faithful LeNet-5)
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 6, 5, padding=2), nn.Tanh(), nn.AvgPool2d(2),
            nn.Conv2d(6, 16, 5), nn.Tanh(), nn.AvgPool2d(2))
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 5 * 5, 120), nn.Tanh(),
            nn.Linear(120, 84), nn.Tanh(),
            nn.Linear(84, 10))

    def forward(self, x):
        return self.classifier(self.features(x))
```

In [5]:
```
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()

        def block(cin, cout):
            return nn.Sequential(
                nn.Conv2d(cin, cout, 3, padding=1),
                nn.BatchNorm2d(cout), nn.ReLU(), nn.MaxPool2d(2))

        self.features = nn.Sequential(block(1, 32), block(32, 64), block(64, 128
        # FIX 3: small hidden FC layer + dropout gives the head more capacity
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 3 * 3, 128), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(128, 10))

    def forward(self, x):
        return self.classifier(self.features(x))
```

In [6]:
```
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    all_preds, all_labels = [], []
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            out = model(x)
            total_loss += criterion(out, y).item() * y.size(0)
            pred = out.argmax(1)
            correct += (pred == y).sum().item()
            total += y.size(0)
```

```
            all_preds.append(pred.cpu())
            all_labels.append(y.cpu())
    return (total_loss / total, 100 * correct / total,
            torch.cat(all_preds), torch.cat(all_labels))


def train_model(model, epochs=10, max_lr=3e-3):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.AdamW(model.parameters(), lr=max_lr, weight_decay=1e-4)
    scheduler = optim.lr_scheduler.OneCycleLR(
        optimizer, max_lr=max_lr, epochs=epochs, steps_per_epoch=len(trainloader
    hist = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}

    for ep in range(epochs):
        model.train()
        for x, y in trainloader:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            loss = criterion(model(x), y)
            loss.backward()
            optimizer.step()
            scheduler.step()          # step per batch for OneCycleLR

        tr_loss, tr_acc, _, _ = evaluate(model, train_eval_loader, criterion)
        te_loss, te_acc, _, _ = evaluate(model, testloader, criterion)
        hist["train_loss"].append(tr_loss); hist["train_acc"].append(tr_acc)
        hist["test_loss"].append(te_loss);  hist["test_acc"].append(te_acc)
        print(f"Epoch {ep+1}/{epochs} | train loss {tr_loss:.4f} acc {tr_acc:.2f
              f"| test loss {te_loss:.4f} acc {te_acc:.2f}%")
    return hist
```

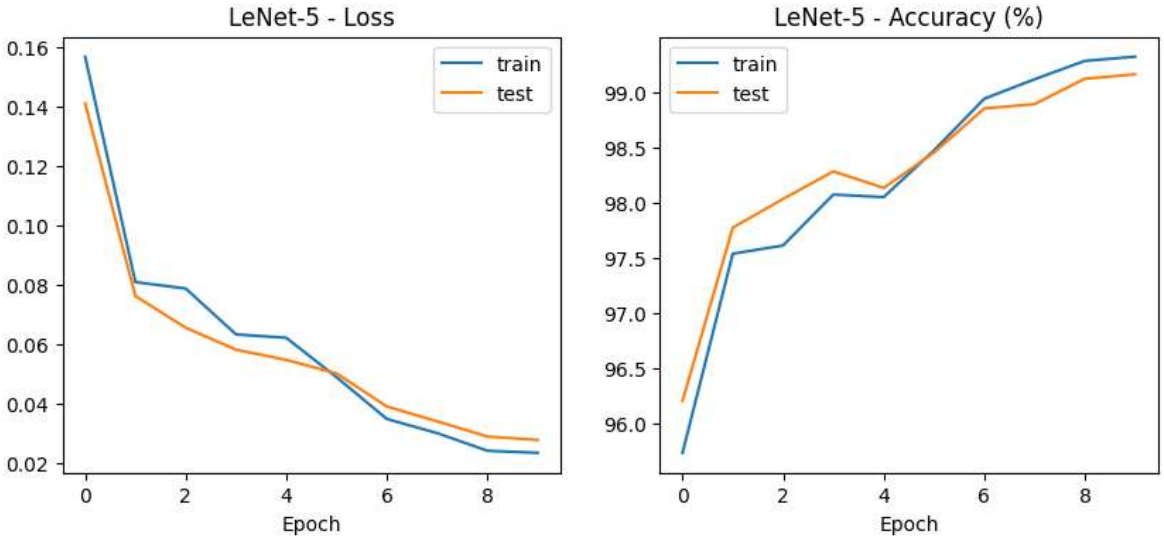
In [7]:

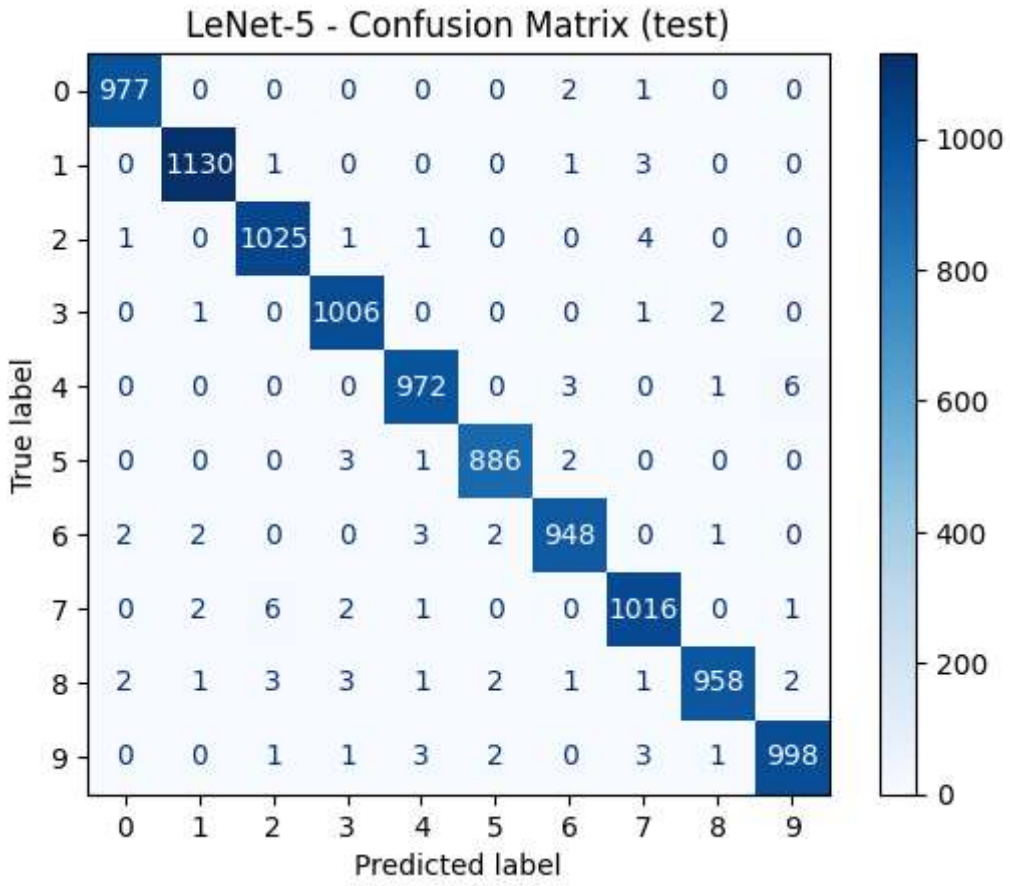
```
def plot_history(hist, name):
    fig, ax = plt.subplots(1, 2, figsize=(10, 4))
    ax[0].plot(hist["train_loss"], label="train"); ax[0].plot(hist["test_loss"],
    ax[0].set_title(f"{name} - Loss"); ax[0].set_xlabel("Epoch"); ax[0].legend()
    ax[1].plot(hist["train_acc"], label="train"); ax[1].plot(hist["test_acc"], l
    ax[1].set_title(f"{name} - Accuracy (%)"); ax[1].set_xlabel("Epoch"); ax[1].
    plt.show()


def show_confusion(model, name):
    _, _, preds, labels = evaluate(model, testloader, nn.CrossEntropyLoss())
    cm = confusion_matrix(labels, preds)
    ConfusionMatrixDisplay(cm, display_labels=range(10)).plot(cmap="Blues")
    plt.title(f"{name} - Confusion Matrix (test)")
    plt.show()


torch.manual_seed(SEED)
lenet = LeNet5()
lenet_hist = train_model(lenet, epochs=10)
plot_history(lenet_hist, "LeNet-5")
show_confusion(lenet, "LeNet-5")
```

```
Epoch 1/10 | train loss 0.1566 acc 95.73% | test loss 0.1409 acc 96.20%
Epoch 2/10 | train loss 0.0808 acc 97.53% | test loss 0.0761 acc 97.77%
Epoch 3/10 | train loss 0.0787 acc 97.61% | test loss 0.0655 acc 98.03%
Epoch 4/10 | train loss 0.0632 acc 98.07% | test loss 0.0581 acc 98.28%
Epoch 5/10 | train loss 0.0621 acc 98.05% | test loss 0.0546 acc 98.13%
Epoch 6/10 | train loss 0.0488 acc 98.47% | test loss 0.0501 acc 98.45%
Epoch 7/10 | train loss 0.0348 acc 98.94% | test loss 0.0390 acc 98.85%
Epoch 8/10 | train loss 0.0300 acc 99.11% | test loss 0.0340 acc 98.89%
Epoch 9/10 | train loss 0.0241 acc 99.28% | test loss 0.0289 acc 99.12%
Epoch 10/10 | train loss 0.0234 acc 99.32% | test loss 0.0277 acc 99.16%
```



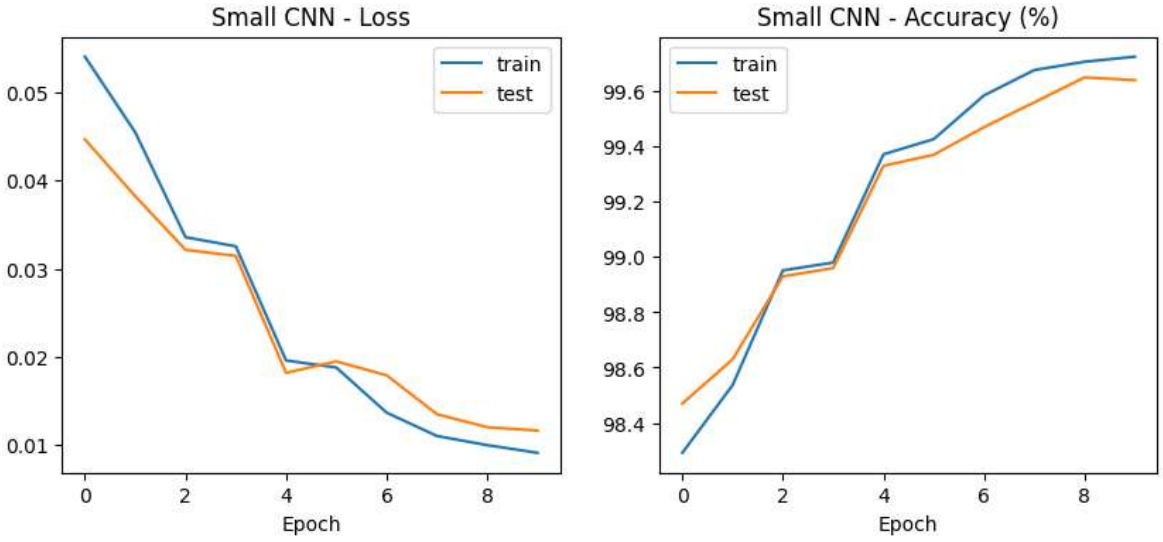


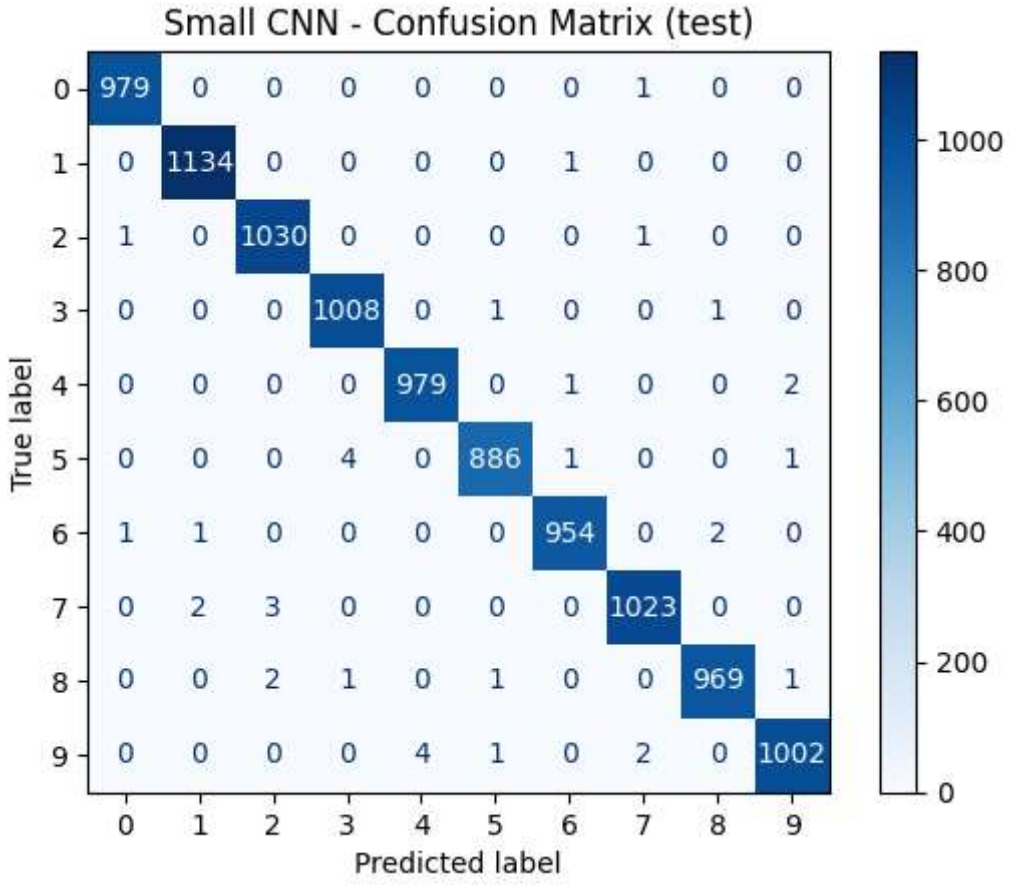
In [8]:

```
torch.manual_seed(SEED)
cnn = SmallCNN()
cnn_hist = train_model(cnn, epochs=10)
plot_history(cnn_hist, "Small CNN")
show_confusion(cnn, "Small CNN")
```

```
print(f"LeNet-5   : train {lenet_hist['train_acc'][-1]:.2f}%, test {lenet_hist['
print(f"Small CNN : train {cnn_hist['train_acc'][-1]:.2f}%, test {cnn_hist['test
```

```
Epoch 1/10 | train loss 0.0540 acc 98.29% | test loss 0.0446 acc 98.47%
Epoch 2/10 | train loss 0.0454 acc 98.54% | test loss 0.0382 acc 98.63%
Epoch 3/10 | train loss 0.0336 acc 98.95% | test loss 0.0321 acc 98.93%
Epoch 4/10 | train loss 0.0325 acc 98.98% | test loss 0.0314 acc 98.96%
Epoch 5/10 | train loss 0.0196 acc 99.37% | test loss 0.0182 acc 99.33%
Epoch 6/10 | train loss 0.0188 acc 99.43% | test loss 0.0195 acc 99.37%
Epoch 7/10 | train loss 0.0137 acc 99.58% | test loss 0.0179 acc 99.47%
Epoch 8/10 | train loss 0.0110 acc 99.68% | test loss 0.0135 acc 99.56%
Epoch 9/10 | train loss 0.0100 acc 99.71% | test loss 0.0120 acc 99.65%
Epoch 10/10 | train loss 0.0091 acc 99.72% | test loss 0.0117 acc 99.64%
```






```
LeNet-5   : train 99.32%, test 99.16%
Small CNN : train 99.72%, test 99.64%
```

(a) LeNet-5 after 10 epochs LeNet5 has two 5×5 convolutions (6 and 16 filters) with Tanh and average pooling, followed by 120-84-10 fully connected layers, about 61.7k parameters in total. I trained it for 10 epochs with AdamW, a OneCycleLR schedule (max lr 3e-3), batch size 64 and random affine augmentation. Accuracy was measured on unaugmented images. Training accuracy: 99.32% (loss 0.0234) Test accuracy: 99.16% (loss 0.0277)

(b) Small CNN under the same conditions The Small CNN has three blocks of 3×3 conv, BatchNorm, ReLU and max-pool (32, 64, 128 filters), then a 128-unit hidden layer with Dropout(0.3) and an output layer. It has about 242k parameters and was trained with the same settings. Training accuracy: 99.72% (loss 0.0091) Test accuracy: 99.64% (loss 0.0117)

(c) Curves and confusion matrices LeNet5 Training and test loss both fall steadily (0.157 to 0.023 and 0.141 to 0.028) and test accuracy rises from 96.2% to 99.16%. There is a small dip at epoch 5 and most of the gain comes in epochs 6-9 as the learning rate decays. The train-test gap is only about 0.16 points so there is no real overfitting and the model was still improving at epoch 10.

Small CNN It starts much stronger at 98.47% test accuracy after epoch 1 (LeNet: 96.2%). It stalls around 99% in epochs 3-4 jumps to 99.33% at epoch 5 and ends at 99.64%. Test loss keeps falling to 0.0117 at epoch 10 and the gap is just 0.08 points. so it generalises very well.

Q3

In [ ]:
```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim import lr_scheduler
import torch.backends.cudnn as cudnn
import numpy as np
import torchvision
from torchvision import datasets, models, transforms
import matplotlib.pyplot as plt
import time
import os
from PIL import Image
from tempfile import TemporaryDirectory

cudnn.benchmark = True
plt.ion()   # interactive mode
```

Out[ ]:
```
<contextlib.ExitStack at 0x1870fec6b00>
```

In [ ]:
```
# Data augmentation and normalization for training
# Just normalization for validation
data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
```

```
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}

data_dir = './hymenoptera_data'
image_datasets = {x: datasets.ImageFolder(os.path.join(data_dir, x),
                                          data_transforms[x])
                  for x in ['train', 'val']}
dataloaders = {x: torch.utils.data.DataLoader(image_datasets[x], batch_size=4,
                                              shuffle=True, num_workers=4)
               for x in ['train', 'val']}
dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val']}
class_names = image_datasets['train'].classes


device = torch.accelerator.current_accelerator().type if torch.accelerator.is_av
print(f"Using {device} device")
```

```
Using xpu device
```

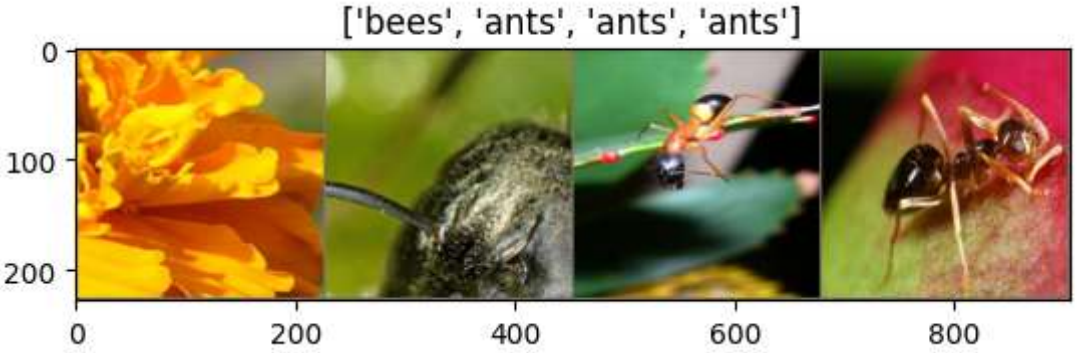
In [3]:

```
def imshow(inp, title=None):
    """Display image for Tensor."""
    inp = inp.numpy().transpose((1, 2, 0))
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    inp = std * inp + mean
    inp = np.clip(inp, 0, 1)
    plt.imshow(inp)
    if title is not None:
        plt.title(title)
    plt.pause(0.001)  # pause a bit so that plots are updated


# Get a batch of training data
inputs, classes = next(iter(dataloaders['train']))

# Make a grid from batch
out = torchvision.utils.make_grid(inputs)

imshow(out, title=[class_names[x] for x in classes])
```



In [4]:

```
def train_model(model, criterion, optimizer, scheduler, num_epochs=25):
    since = time.time()

    # Create a temporary directory to save training checkpoints
    with TemporaryDirectory() as tempdir:
        best_model_params_path = os.path.join(tempdir, 'best_model_params.pt')

        torch.save(model.state_dict(), best_model_params_path)
        best_acc = 0.0

        for epoch in range(num_epochs):
            print(f'Epoch {epoch}/{num_epochs - 1}')
            print('-' * 10)

            # Each epoch has a training and validation phase
            for phase in ['train', 'val']:
                if phase == 'train':
                    model.train()  # Set model to training mode
                else:
                    model.eval()   # Set model to evaluate mode

                running_loss = 0.0
                running_corrects = 0

                # Iterate over data.
                for inputs, labels in dataloaders[phase]:
                    inputs = inputs.to(device)
                    labels = labels.to(device)

                    # zero the parameter gradients
                    optimizer.zero_grad()

                    # forward
                    # track history if only in train
                    with torch.set_grad_enabled(phase == 'train'):
                        outputs = model(inputs)
                        _, preds = torch.max(outputs, 1)
                        loss = criterion(outputs, labels)

                        # backward + optimize only if in training phase
                        if phase == 'train':
                            loss.backward()
                            optimizer.step()

                    # statistics
                    running_loss += loss.item() * inputs.size(0)
                    running_corrects += torch.sum(preds == labels.data)
                if phase == 'train':
                    scheduler.step()

                epoch_loss = running_loss / dataset_sizes[phase]
                epoch_acc = running_corrects.double() / dataset_sizes[phase]

                print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

                # deep copy the model
                if phase == 'val' and epoch_acc > best_acc:
                    best_acc = epoch_acc
                    torch.save(model.state_dict(), best_model_params_path)
```

```
            print()

        time_elapsed = time.time() - since
        print(f'Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 6
        print(f'Best val Acc: {best_acc:4f}')

        # load best model weights
        model.load_state_dict(torch.load(best_model_params_path, weights_only=Tr
    return model
```

In [5]:
```
def visualize_model(model, num_images=6):
    was_training = model.training
    model.eval()
    images_so_far = 0
    fig = plt.figure()

    with torch.no_grad():
        for i, (inputs, labels) in enumerate(dataloaders['val']):
            inputs = inputs.to(device)
            labels = labels.to(device)

            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)

            for j in range(inputs.size()[0]):
                images_so_far += 1
                ax = plt.subplot(num_images//2, 2, images_so_far)
                ax.axis('off')
                ax.set_title(f'predicted: {class_names[preds[j]]}')
                imshow(inputs.cpu().data[j])

                if images_so_far == num_images:
                    model.train(mode=was_training)
                    return
        model.train(mode=was_training)
```

Part a - AS Fine tuning model

In [11]:
```
model_ft = models.resnet18(weights='IMAGENET1K_V1')
num_ftrs = model_ft.fc.in_features
# Here the size of each output sample is set to 2.
# Alternatively, it can be generalized to ``nn.Linear(num_ftrs, len(class_names)
model_ft.fc = nn.Linear(num_ftrs, 2)

model_ft = model_ft.to(device)

criterion = nn.CrossEntropyLoss()

# Observe that all parameters are being optimized
optimizer_ft = optim.SGD(model_ft.parameters(), lr=0.001, momentum=0.9)

# Decay LR by a factor of 0.1 every 7 epochs
exp_lr_scheduler = lr_scheduler.StepLR(optimizer_ft, step_size=7, gamma=0.1)
```

In [12]:
```
model_ft = train_model(model_ft, criterion, optimizer_ft, exp_lr_scheduler,
                       num_epochs=25)
```

```
Epoch 0/24
----------
train Loss: 0.6646 Acc: 0.6926
val Loss: 0.3244 Acc: 0.8562

Epoch 1/24
----------
train Loss: 0.5111 Acc: 0.7869
val Loss: 0.2745 Acc: 0.9216

Epoch 2/24
----------
train Loss: 0.4807 Acc: 0.7992
val Loss: 0.4167 Acc: 0.8627

Epoch 3/24
----------
train Loss: 0.3660 Acc: 0.8443
val Loss: 0.2140 Acc: 0.9020

Epoch 4/24
----------
train Loss: 0.4761 Acc: 0.8074
val Loss: 0.1914 Acc: 0.9281

Epoch 5/24
----------
train Loss: 0.4184 Acc: 0.8402
val Loss: 0.1833 Acc: 0.9281

Epoch 6/24
----------
train Loss: 0.4575 Acc: 0.8033
val Loss: 0.7331 Acc: 0.7974

Epoch 7/24
----------
train Loss: 0.3831 Acc: 0.8525
val Loss: 0.2287 Acc: 0.9020

Epoch 8/24
----------
train Loss: 0.2943 Acc: 0.8607
val Loss: 0.1938 Acc: 0.9216

Epoch 9/24
----------
train Loss: 0.2660 Acc: 0.9016
val Loss: 0.2129 Acc: 0.9085

Epoch 10/24
----------
train Loss: 0.2718 Acc: 0.8975
val Loss: 0.1676 Acc: 0.9216

Epoch 11/24
----------
train Loss: 0.2324 Acc: 0.9098
val Loss: 0.1583 Acc: 0.9412
```

```
Epoch 12/24
----------
train Loss: 0.2571 Acc: 0.8852
val Loss: 0.1605 Acc: 0.9346

Epoch 13/24
----------
train Loss: 0.3366 Acc: 0.8484
val Loss: 0.1848 Acc: 0.9150

Epoch 14/24
----------
train Loss: 0.2639 Acc: 0.8975
val Loss: 0.1660 Acc: 0.9281

Epoch 15/24
----------
train Loss: 0.3187 Acc: 0.8648
val Loss: 0.1665 Acc: 0.9346

Epoch 16/24
----------
train Loss: 0.3057 Acc: 0.8525
val Loss: 0.1621 Acc: 0.9346

Epoch 17/24
----------
train Loss: 0.2668 Acc: 0.8934
val Loss: 0.1652 Acc: 0.9281

Epoch 18/24
----------
train Loss: 0.2313 Acc: 0.9098
val Loss: 0.1764 Acc: 0.9412

Epoch 19/24
----------
train Loss: 0.2120 Acc: 0.9180
val Loss: 0.1640 Acc: 0.9412

Epoch 20/24
----------
train Loss: 0.2162 Acc: 0.9098
val Loss: 0.1984 Acc: 0.9020

Epoch 21/24
----------
train Loss: 0.3213 Acc: 0.8484
val Loss: 0.1733 Acc: 0.9281

Epoch 22/24
----------
train Loss: 0.1671 Acc: 0.9467
val Loss: 0.1622 Acc: 0.9412

Epoch 23/24
----------
train Loss: 0.2701 Acc: 0.8893
val Loss: 0.1963 Acc: 0.9085
```

```
Epoch 24/24
----------
train Loss: 0.1822 Acc: 0.9262
val Loss: 0.1756 Acc: 0.9150

Training complete in 9m 45s
Best val Acc: 0.941176
```

In [13]:
```
visualize_model(model_ft)
```

predicted: bees



predicted: ants



predicted: ants



predicted: ants



predicted: ants



predicted: bees



Part b - AS Fixed feature extractor

In [8]:
```
model_conv = torchvision.models.resnet18(weights='IMAGENET1K_V1')
for param in model_conv.parameters():
    param.requires_grad = False

# Parameters of newly constructed modules have requires_grad=True by default
num_ftrs = model_conv.fc.in_features
model_conv.fc = nn.Linear(num_ftrs, 2)

model_conv = model_conv.to(device)

criterion = nn.CrossEntropyLoss()

# Observe that only parameters of final layer are being optimized as
# opposed to before.
optimizer_conv = optim.SGD(model_conv.fc.parameters(), lr=0.001, momentum=0.9)

# Decay LR by a factor of 0.1 every 7 epochs
exp_lr_scheduler = lr_scheduler.StepLR(optimizer_conv, step_size=7, gamma=0.1)
```

In [9]:
```
model_conv = train_model(model_conv, criterion, optimizer_conv,
                         exp_lr_scheduler, num_epochs=25)
```

```
Epoch 0/24
----------
train Loss: 0.5886 Acc: 0.6844
val Loss: 0.2878 Acc: 0.8758

Epoch 1/24
----------
train Loss: 0.6317 Acc: 0.6967
val Loss: 0.1695 Acc: 0.9412

Epoch 2/24
----------
train Loss: 0.4886 Acc: 0.7746
val Loss: 0.2374 Acc: 0.9020

Epoch 3/24
----------
train Loss: 0.3155 Acc: 0.8689
val Loss: 0.2531 Acc: 0.9085

Epoch 4/24
----------
train Loss: 0.5081 Acc: 0.8156
val Loss: 0.2857 Acc: 0.8954

Epoch 5/24
----------
train Loss: 0.5929 Acc: 0.7664
val Loss: 0.1821 Acc: 0.9346

Epoch 6/24
----------
train Loss: 0.4583 Acc: 0.8074
val Loss: 0.2489 Acc: 0.9085

Epoch 7/24
----------
train Loss: 0.4004 Acc: 0.8525
val Loss: 0.2229 Acc: 0.9216

Epoch 8/24
----------
train Loss: 0.3957 Acc: 0.8279
val Loss: 0.2100 Acc: 0.9216

Epoch 9/24
----------
train Loss: 0.3572 Acc: 0.8484
val Loss: 0.2455 Acc: 0.8954

Epoch 10/24
----------
train Loss: 0.3765 Acc: 0.8156
val Loss: 0.1939 Acc: 0.9412

Epoch 11/24
----------
train Loss: 0.3848 Acc: 0.8115
val Loss: 0.1853 Acc: 0.9281
```

```
Epoch 12/24
----------
train Loss: 0.2879 Acc: 0.8934
val Loss: 0.1695 Acc: 0.9412

Epoch 13/24
----------
train Loss: 0.4121 Acc: 0.8156
val Loss: 0.1787 Acc: 0.9412

Epoch 14/24
----------
train Loss: 0.3897 Acc: 0.8279
val Loss: 0.1956 Acc: 0.9281

Epoch 15/24
----------
train Loss: 0.3519 Acc: 0.8443
val Loss: 0.1997 Acc: 0.9216

Epoch 16/24
----------
train Loss: 0.2837 Acc: 0.8770
val Loss: 0.1670 Acc: 0.9412

Epoch 17/24
----------
train Loss: 0.3619 Acc: 0.8361
val Loss: 0.1731 Acc: 0.9412

Epoch 18/24
----------
train Loss: 0.3841 Acc: 0.8402
val Loss: 0.1823 Acc: 0.9412

Epoch 19/24
----------
train Loss: 0.3464 Acc: 0.8443
val Loss: 0.2840 Acc: 0.8889

Epoch 20/24
----------
train Loss: 0.4044 Acc: 0.8320
val Loss: 0.1820 Acc: 0.9346

Epoch 21/24
----------
train Loss: 0.3089 Acc: 0.8361
val Loss: 0.1947 Acc: 0.9281

Epoch 22/24
----------
train Loss: 0.3104 Acc: 0.8648
val Loss: 0.1952 Acc: 0.9150

Epoch 23/24
----------
train Loss: 0.2792 Acc: 0.8811
val Loss: 0.1958 Acc: 0.9346
```

```
Epoch 24/24
----------
train Loss: 0.3182 Acc: 0.8525
val Loss: 0.1843 Acc: 0.9346

Training complete in 7m 36s
Best val Acc: 0.941176
```

In [10]:
```
visualize_model(model_conv)

plt.ioff()
plt.show()
```

predicted: ants



predicted: ants



predicted: bees



predicted: bees



predicted: bees



predicted: ants



Fine tuning and fixed feature extraction represent two distinct approaches to transfer learning using pre trained models like ResNet18. In fine tuning the entire network has its weights updated with no layers frozen. It meaning the optimizer targets all model parameters and this results in slower training speeds and higher memory consumption because gradients must be computed and backpropagated through every layer. Fixed feature extraction freezes all backbone layers and trains only the newly initialized final classification layer by passing solely the final layer parameters to the optimizer. This approach is much faster and consumes less memory due to minimal gradient storage requirements.